# SRS - Software Requirements Specification

This document follows the IEEE Software Requirements Specification based on the superceded standard 830 - 1998.
For readability, only the male form is used in this explanatory text in order to represent female and male persons.

http://www.cse.msu.edu/~cse870/IEEEXplore-SRS-template.pdf

## 1. Introduction

This Section introduces the project MailVote of Polytech Nantes. The goal is to provide an automatic assignment interface in which users can provide their preference on a number of choices before the system automatically assigns each user to a choice. The interface is similar to mailman's list management interface, for further details, please refer to [1].

### 1.1 Purpose

A limited number of persons (CLIENTS) will be assigned to a limited number of choices (CHOICES). Each CLIENT will be able to express his preference on the CHOICES by using a predefined number of points (TOKENS) per CLIENT. The system is entirely based on emails that are stored in an IMAP mail server. This concerns the configuration (CONFIGURATION) of the system including the CLIENT's email addresses and the CHOICES for a particular assignment (RUN) as well as the decision (DECISION) on which CLIENT will be assigned to which CHOICE.

### 1.2 Scope

The scope of the project is limited to a JAVA application that runs on a JRE of at least version 6. This JAVA application communicates with the IMAP mail server in order to perform its task. It handles all incoming mails that are sent to the email account and sends mails through the IMAP mail server. The user interaction is performed through the mail server. Sophisticated graphical user interfaces are out of the scope of this project. Installation and maintenance of the mail server are outside of the scope of the project.

### 1.3 Definitions, acronyms, and abbreviations

The following list is provided in alphabetical order:

CHOICE: A description of an alternative that CLIENTS may choose from. Each CHOICE is assigned a unique number in a RUN.

CLIENT: The true name and the email address of a participant in the RUN. They are identified by a unique number.

CONSTRAINT: Constraints link CHOICES to CLIENTS. They may either be positive (i.e. a particular CHOICE is a priori assigned to a particular CLIENT), or negative (i.e. a particular CHOICE is not available for a particular CLIENT).

DECISION: After the SELECTION, the DECISION table contains a mapping of each CLIENT to a CHOICE. If a unique mapping is not possible, the system detects an incomplete DECISION.

FOLLOWER: A FOLLOWER is a CLIENT who delegates his choice to another CLIENT. This is particularly useful if two or more CLIENTS would like to express their CHOICE together. The FOLLOWER can be unidirectional or bidirectional, the first CLIENT in a FOLLOWER group who provides a CHOICE defines the CHOICE of the FOLLOWER group.

INITIATOR: The person(s) who has/have sent the initiation email for a particular RUN. Each RUN may have a different INITIATOR. INITIATORS have the role of administrators, i.e. they may add CHOICES, manage the list of CLIENTS, and launch the SELECTION at any time.

RUN: A single assignment of CLIENTS to CHOICES started by INITIATORS and ending in a DECISION.

SELECTION: The process of automatic assignment of CLIENTS to CHOICES. It results in a (complete or incomplete) DECISION.

TOKEN: A virtual “coin” that a CLIENT may put on a CHOICE that he prefers. Each CLIENT has an equal and predefined number of TOKENS that he may distribute on the CHOICES of his preference.

## 1.4 References

[1] GNU Foundation, “GNU Mailman List Member Manual Email commands quick reference”, available online: http://www.list.org/mailman-member/node41.html, last updated: March 2, 2015, Last accesssed: 18 March 2017.

[2] IEEE-SA Standards Board, “IEEE Recommended Practice for Software Requirements Specifications”, iEEE Std 830-1998(R2009), 2009

## 1.5 Overview

Chapter 2 describes the software product in further detail from the viewpoint of the user and the surrounding system architecture. In Chapter 3, a developer centered perspective is chosen.

# 2. Overall description

## 2.1 Product perspective

MailVote is strongly linked to an IMAP mail server architecture. In the IMAP mail box, folders and subfolders will be created in order to sort the incoming and outgoing mails with regard to their usage in the system. The IMAP mail server will be used for communicating with the users, i.e. INITIATORS and CLIENTS.
The user interface is limited to interpreting mails sent from the CLIENTS and INITIATORS that contain keywords in a specific syntax as defined in Section 3.1.1.
The software shall run on a JAVA compatible machine with 1GB of free memory. On a Sony Vaio VPCSE notebook, the execution time shall not exceed 5 minutes for reading 5 configured RUNS, performing 1 SELECTION, and sending 1 DECISION.

## 2.2 Product functions

From the user perspective, the simplest process is divided into several phases:

Phase 1:
- One of the INITIATORS sends one email to the IMAP server containing the name of the RUN, its DESCRIPTION, the number of TOKENS per CLIENT, and the names and email addresses of the INITIATORS.
- The system responds with a confirmation mail sent to all INITIATORS.

Phase 2:
- The INITIATORS send the list of CHOICES in one or several mails.
- The INITIATORS send the list of CLIENTS in one or several mails.
- The INITIATORS ask the system to send the invitation to the clients.
- The system informs each CLIENT about the DESCRIPTION of the RUN and his possibilities of performing a vote for the CHOICES or of becoming a FOLLOWER.

Phase 3:
- The CLIENTS send their votes for the CHOICES using their TOKENS or declare themselves as FOLLOWERS.
- The INITIATORS ask the system for SELECTION.
- The system responds by assigning the CHOICES to the CLIENTS such that their preferences expressed by the TOKENS are respected, i.e. by a current status on the DECISION.

Phase 4:
- The INITIATORS ask the system to communicate the DECISION to the CLIENTS.

## 2.3 User characteristics

The users need to be able to send and receive emails. They also need to be able to understand the formal help text provided by the system. A typical user is an engineering student at the University of Nantes after the first semester.

## 2.4 Constraints

The software has to be written in JAVA using an object oriented design. The condition that several instances of the same software work on the same IMAP account shall be avoided by a locking mechanism. The lock shall expire after a timeout.

## 2.5 Assumptions and dependencies

It is assumed that the specified protocol for voting is complete. Changes to this SRS are required if the procedure of voting cannot be successfully completed by the specified protocol.

# 3. Specific requirements organized by feature

This section contains detailed information about the features of the software described from a developer perspective.

## 3.1 External interface requirements

The configuration of the IMAP account used by the software shall be specified in a configuration file or on the command line. The configuration file shall be editable by a text editor. XML, config style, or similar may be chosen by the developers but require validation by the client.
The IMAP account configuration shall be the only information stored by the software. In particular, no interpreted data shall be stored locally. If such storage is required, it shall be stored in an email on the IMAP server.

### 3.1.1 User interfaces

The user interface is performed by sending emails to the IMAP mail server. The following protocol is defined. It is provided in exactly the format that shall be returned by the system to the user (CLIENT or INITIATOR) when a HELP message is received. The system shall detect a command as an exact case-insensitive match starting with the first alphanumerical character of the subject line or starting with the first alphanumerical character of any line in the body of the message. Valid and invalid examples are provided in the Annex.
The HELP message is defined as follows:

--- start help message ---

Dear user,
This system provides an interface for voting on a predefined number of choices using a token based voting system. After a voting procedure is configured, clients may assign a number of tokens to each choice of their preference in order to express their opinion. The system automatically assigns each client to one of the choices.
The vote is set up by an initiator. The initiator creates the run and configures its parameters. He then invites the clients. The syntax for registering initiators and clients is defined as ADDRESS, containing three fields: Name Surname Mailaddress. An example for ADDRESS is “Leonardo DaVinci Leonardo.Davinci@someserver.com”. The syntax for identifying registered runs, choices, clients and initiators is IDENTIFIER which is a random unique 64bit number assigned by the system.
If not stated otherwise, the system answers with a STATUS message to a successfully processed request and with an error message detailing the error otherwise.

===
General Commands:
===
HELP
This help message is returned.

RUN IDENTIFIER
Selects a specific run with the given identifier. Shall always precede any other command except CREATERUN and HELP.

USER IDENTIFIER
Selects the client or initiator by its identifier.

STATUS
Returns the status of the current evaluation for this particular client or initiator. The response for a client contains the configuration parameters of the RUN, the client’s CHOICE, and potential FOLLOWER information. The response for an initiator contains the configuration parameters for the RUN, the list of all configured CLIENTS and their CHOICES, and the current DECISION.

===
Client Commands:
===
VOTE IDENTIFIER, NUMBER_OF_TOKENS
The client votes for a particular CHOICE by placing a number of tokens on that choice’s identifier. One or several VOTE commands are sent in one email. The total number of tokens in one email has to match the number of tokens configured by the initiator. A client may send several mails to the system when his opinion on the preferences changes. A STATUS is returned when the system accepts the choice. An error message is returned if the total number of tokens does not match the configuration, when the identifiers specified are invalid

for this run, or when the client is a FOLLOWER of another client who provided a vote email first.

FOLLOW IDENTIFIER
The client delegates the voting to another client that is specified in the identifier. The identifier has to be communicated to the follower by the other client or by an initiator.

===
Initiator Commands:
===

CREATERUN ADDRESS
Creates and selects a new run. All further commands are configuring the created run. The system answers with the initiators version of STATUS.

TOKENCOUNT number
The number of tokens that a client holds in order to express his preference in this run.

DESCRIPTION text
The description of the run as presented to the clients.

ADDCLIENT ADDRESS
Adds a client to the current run.

DELCLIENT identifier
Deletes a client from the current run.

ADDCHOICE text
Adds the provide text of the choice to the list of choices.

DELCHOICE identifier
Deletes the choice corresponding to the identifier from the list of choices.

SENDINVITATION text
Sends an invitation to all clients providing the additional text in the email, i.e. for identifying the initiator who send the text or for specifying a voting deadline. This invitation contains the description of the current run and instructions on how to perform the voting with preconfigured commands for the answer. As the client may use the reply button in order to perform his vote, the text of the message is written such that the interpretation of the system conforms to the desired effect of voting.

SENDDECISION text
Sends a decision mail to all clients, informing each client which choice was assigned to him. The additional text is sent to the clients in the mail. This usually finishes a run although clients

may continue voting and a several decision mails may be sent by the initiator in which case, text may be used to identify draft and final decisions.

--- end help message ---

### 3.1.2 Hardware interfaces

The software shall not rely on the availability of a graphical terminal, i.e. there shall be no graphical user interface.

### 3.1.3 Software interfaces

### 3.1.4 Communications interfaces

## 3.2 System features

### 3.2.1 System Feature 1

#### 3.2.1.1 Introduction/Purpose of feature

#### 3.2.1.2 Stimulus/Response sequence

#### 3.2.1.3 Associated functional requirements

##### 3.2.1.3.1 Functional requirement 1

### 3.2.2 System Feature 2

#### 3.2.2.1 Introduction/Purpose of feature

#### 3.2.2.2 Stimulus/Response sequence

#### 3.2.2.3 Associated functional requirements

#### 3.2.2.3.1 Functional requirement 1

### 3.3 Performance requirements

### 3.4 Design constraints

### 3.5 Software system attributes

### 3.6 Other requirements

# Annex

## Annex A. Valid and invalid examples

The following are valid examples for sending the XYZ command:

Subject: XYZ
Body:
<empty>

Subject: Please provide me with some information
Body:
Dear system,
Please execute my demand
XYZ
Best regards,

Subject: System message from voting system
Body:
> Dear user,
> this is a voting system. You can simply execute the command XYZ as follows:

> XYZ
> Best regards,

The following are invalid examples and shall be answered by the system with the HELP text:
Subject: Please execute XYZ
Body: <empty>

Subject: To the system
Body:
Dear system,
please execute my demand in the following order
1) XYZ
Best regards,